


[Mission](#)
[Editorial Committee](#)
[Process and Structure](#)
[Code4Lib](#)

Issue 58, 2023-12-04

Pipeline or Pipe Dream: Building a Scaled Automated Metadata Creation and Ingest Workflow Using Web Scraping Tools

Since 2004, the FRASER Digital Library has provided free access to publications and archival collections related to the history of economics, finance, banking, and the Federal Reserve System. The agile web development team that supports FRASER's digital asset management system embarked on an initiative to automate collecting documents and metadata from US governmental sources across the web. These sources present their content on web pages but do not serve the metadata and document links via an API or other semantic web technologies, making automation a unique challenge. Using a combination of third-party software, lightweight cloud services, and custom Python code, the FRASER Recurring Downloads project transformed what was previously a labor-intensive daily process into a metadata creation and ingest pipeline that requires minimal human intervention or quality control.

This article will provide an overview of the software and services used for the Recurring Downloads pipeline, as well as some of the struggles that the team encountered during the design and build process, and current use of the final product. The project required a more detailed plan than was designed and documented. The fully manual process was not intended to be automated when established, which introduced inherent complexity in creating the pipeline. A more comprehensive plan could have made the iterative development process easier by having a defined data model, and documentation of—and strategy for—edge cases. Further initial analysis of the cloud services used would have defined the limitations of those services, and workarounds could have been accounted for in the project plan. While the labor-intensive manual workflow has been reduced significantly, the required skill sets to efficiently maintain the automated workflow present a sustainability challenge of task distribution between librarians and developers. This article will detail the challenges and limitations of transitioning and standardizing recurring web scraping across more than 50 sources to a semi-automated workflow and potential future improvements to the pipeline.

by Matthew Krc and Anna Oates Schlaack

Introduction

The Federal Reserve Bank of St. Louis's FRASER Digital Library is composed of historical and contemporary publications, as well as other types of digitized and born-digital content that are curated and maintained by a team of librarians and supporting staff. The digital library staff capture and archive contemporary digital publications on a recurring basis as they are issued. These digital publications, like myriad information on the web that have archival value, are not presented as structured data that is easily retrievable through standard automated processes—e.g., metadata capture via an application programming interface (API). An institution that wishes to continuously collect such documents must either depend upon frequent, manual checks of source websites or on services that monitor and identify changes to a given web page. A challenge to automating this archival process of checking for new content is having a reliable system that can capture digital resources and readily available metadata with minimal human intervention. Despite this core challenge and other potential obstacles, in 2022, the FRASER Digital Library team embarked on a project to attempt automating elements of, what had previously been, a tedious, manual process.

Background

Since 2004, the [FRASER Digital Library](#) has provided free access to publications and archival collections related to the history of economics, finance, banking, and the Federal Reserve System. The collections are maintained by a team of four library staff, and the digital library portal and digital asset management system (DAMS) are maintained by a team of three web developers. The library services staff and information technology staff ultimately report to the same division head, though through different reporting structures, and the two teams work together in an agile project management environment to address enhancements and maintenance for the application.

Physical collections for in-house or outsourced digitization are obtained through partnerships with external host repositories. Digital content is primarily collected directly from government institutions' websites. In 2006, digital library staff began collecting content as it was published [1]. Over the years, the manual collection workflow saw several iterations to make the process more efficient. In 2015, staff began using [Distill](#), a tool that monitors web pages and sends alerts when a web page changes. This software was particularly valuable for reducing the extensive time required to survey sources for new content. While this improved the process by reducing the time required to evaluate whether a source had published new content, automated monitoring of web page changes was not a perfect process. The majority of alerts were false positives—i.e., Distill identified that there was a change to the web page, but there was no new content. For example, if an icon moved to another location on the page or if new tweets appeared in the embedded Twitter feed, the tool would send an alert. Between February 2018 – July 2022, 46% of web page change alerts were false positives—resulting in additional time to filter out alerts which did produce new content.

FRASER collections range from government documents and data releases to academic publications and personal archives. Each of those content types require different metadata and, in the case of the contemporary digital publications, the descriptive metadata provided by the source varies. Some publications in FRASER's collection scope are captured from the Government Publishing Office's (GPO) [GovInfo](#), which serves rich, standardized metadata. Other publications, such as the [Press Releases of the U.S. Department of the Treasury](#), present only a title and date of publication on the source website. In addition to the challenge of disparate metadata availability, the unique publication schedules posed challenges to the manual workflow and any potential automation. The Distill web monitoring tool was so effective for this reason—it offered the possibility of knowing when an irregular

publication had been issued simply by checking for changes on the page. Albeit with faults, automated monitoring streamlined the process of tracking when sources had been checked for new content. Timeliness to ingest new publications in FRASER is key to ensuring that the digital library is a trustworthy location for finding historic and contemporary documents on economics. As seen with the web monitoring tool, any attempt at automation could be thwarted by the possibility of dynamic websites constantly changing.

As of 2021, the number of publications included in the recurring downloads workflow had increased significantly [2], and, with that growth, the time required to monitor alerts, collect, describe, and ingest content manually as it was published became a full-time position. To alleviate staff time required to complete the largely monotonous workflow, in 2019, digital library staff began exploring possibilities for automation. The most successful attempt at automation was a Python program that ran daily to scrape a web page, check to see whether the captured elements already existed in the DAMS, and, if not, transform and ingest the metadata and file, and send an email notification to staff when the ingest was complete and ready to be made public. The program was customized for a single publication that was the most laborious to capture because of its high publication frequency, with new content published as often as every 15 minutes [3]. Given the aforementioned challenges of variance in sources, automating harvesting for all of the publications was not achievable with a simple program but required an extensible web scraping framework.

Recurring Downloads Design Phase

Informed by the successful Python program pilot, a team of architects and the agile web development team that supports FRASER began a design phase to ideate a pipeline that would scrape metadata and files from a set of more than 50 websites, transform and enhance the metadata, and handle ingest of the metadata and files into the DAMS. The automated recurring downloads process was broken into two components: data processing and data management. Data processing would run entirely in the background and would be managed through a dashboard in the DAMS. The data processing had to be capable of:

- Harvesting and transforming metadata to the local Metadata Object Description Schema (MODS) implementation, following title-level metadata requirements
- Handling when new content is available and filtering out existing content

The ideal data management dashboard would include:

- A list of new records queued for quality assurance (QA)
- Notifications for records missing required metadata fields or with metadata conflicts (e.g., the author of a work does not correspond to an existing name authority record)
- An interface in which to modify existing sources and add new sources to the pipeline
- An interface for selecting elements on a source web page that should be scraped
- An interface to schedule scrape jobs at the title-level (i.e., distinct scraping schedules for each publication)

From the initial design sessions, the team tentatively decided on an infrastructure that:

- Leveraged the [OpenRefine API](#) to handle metadata transformations so that digital library staff had ownership of modifying metadata transformations without requiring changes to the application code
- Employed web scraping software that had out-of-the-box, point-and-click functions for selecting web elements to scrape
- Used [AWS DynamoDB](#) for select administrative data that would be created exclusively for the automated workflow

As evident in the many learning resources, including a [Library Carpentry lesson](#) and [Library Juice Academy course](#), [OpenRefine](#) is a tool widely used in libraries for metadata work, and is invaluable specifically for metadata normalization. With the pipeline, library staff hoped to automate repetitive tasks, including normalizing dates and formatting author names, so that metadata records would be public-ready when ingested into FRASER. The [OpenRefine API](#) enables clients to run automatically a set of OpenRefine operations, functionally fulfilling the transformation operations needed for the pipeline. A set of operations can be created in the OpenRefine user interface, with which FRASER library staff were already familiar. In this proposed pipeline implementation, library staff would create operations in the OpenRefine application, load the operations for use in the pipeline, and assign operations at the title-level. This would enable library staff to add and modify metadata transformation operations without requiring web developer support.

Prior to the recurring downloads project design phase, digital library staff identified [ParseHub](#) and [Web Scraper](#) as two web scraping tools that are easy to use for library staff who do not have coding experience. Building a custom web scraping tool that replicated the same functionality and user-friendly interface would not have added notable value and would have required more time for planning and investing more development resources to build. Web Scraper was ultimately selected for handling the web scraping element of the pipeline because it was the easiest output to process and was configurable to authenticate with other tools.

In order to store metadata transformation templates that would be quickly and easily retrievable, the development team chose to use [AWS DynamoDB](#) for the ease with which users could access and add and edit DynamoDB documents, or database records. DynamoDB is an Amazon Web Services (AWS) managed NoSQL, non-relational database service that allows fast access to stored data. Because the recurring downloads pipeline wouldn't require complex queries on data that would be retrieved, storing the metadata transformation templates in the primary PostgreSQL relational database was an unnecessarily complex approach. Prior to the project discussions, the web development team had been interested in investigating DynamoDB as a more lightweight option for certain data types, and this project was an optimal pilot. In the AWS console, librarians also had access to edit DynamoDB data in a manner more intuitive than typical SQL database clients allow.

Edit item

FormJSON view

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Attributes

Add new attribute ▼

Attribute name	Value	Type	
id - Partition key	title#5158	String	
sortkey - Sort key	title#5158	String	
cron	0 22 * 2,5,8,11 5	String	Remove
filenames	Insert a field ▼	Map	Remove
pdf	treasury_hamp_report_{yyyy}_{q}.pdf	String	Remove
frequency	q	String	Remove
metadata	Insert a field ▼	Map	Remove
s3_path	/docs/historical/fct/treasury/	String	Remove

CancelSaveSave and close

Figure 1. Screenshot of AWS DynamoDB record form in AWS console.

OpenRefine, Web Scraper, and DynamoDB were all new tools to the development team, and integrating OpenRefine in the pipeline was ultimately discarded because its integration in the pipeline would require additional developer analysis, planning, experimentation, and testing. Ultimately, it was easier and, at the time, deemed more cost effective for the web developers to work with a familiar tool rather than learn to integrate one that wasn't native to their existing webstack or a provided AWS service. Instead, digital library staff compiled a list of known metadata transformation operations required, which the development team would integrate into the new workflow as custom code to run in an AWS serverless Lambda service.

Pipeline Implementation

The recurring downloads pipeline was deployed in October 2022. Due to timeline and resource constraints, the architecture and data flow were slightly different and pared down from the finalized design. The lack of some features removed from the released pipeline have posed pain-points for the digital library staff, which are detailed in the [Challenges](#) section of this article. The final pipeline consisted of tools that fulfilled the central function of capturing files and metadata from a source, transforming the files and metadata to follow FRASER requirements, and ingesting the files and metadata to the DAMS for QA before being made public.

Table 1. List of enhancements to existing infrastructure for the metadata pipeline

Tools	Function(s)
Web Scraper Browser Extension (Chrome extension , Firefox add-on)	sitemap creation
Web Scraper Cloud	sitemap storage, scrape job scheduling, scrape job output, metadata preprocessing
AWS DynamoDB	metadata transformation template storage
AWS Lambda	metadata transformation, file transformation
Enhancement to DAMS	new record queuing, record QA

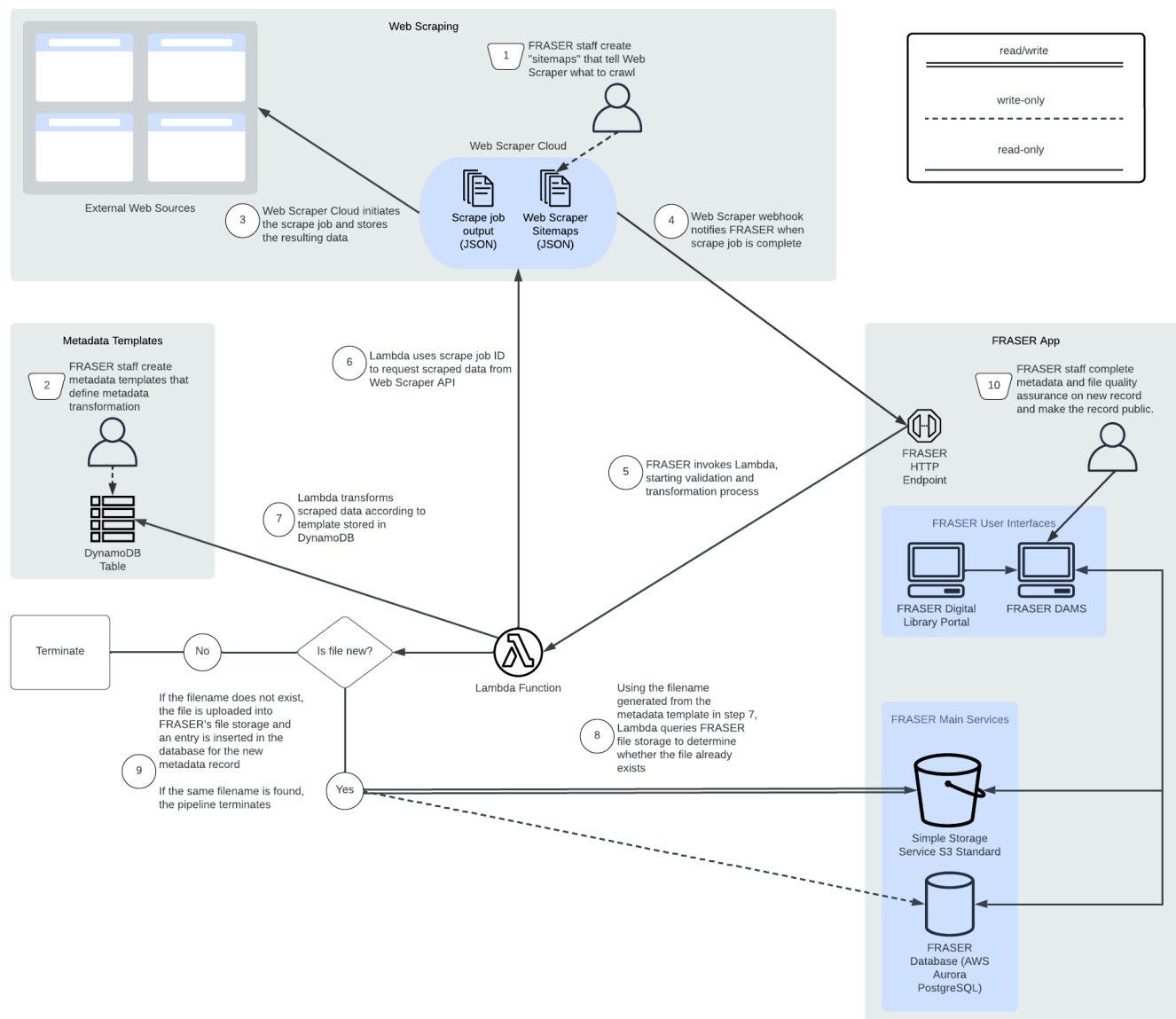


Figure 2. Diagram of recurring downloads pipeline architecture and workflow.

Central to the Web Scraper service are sitemaps. Distinct from [XML Sitemaps](#) that aid search engines in web crawling, Web Scraper sitemaps are JSON documents that define the website that should be scraped with a "startUrl" parameter and what to scrape from the given website with "selectors." Selectors contain information that inform Web Scraper what and how to scrape. To set up a new source to be included in the pipeline, library staff create sitemaps using the Web Scraper Browser Extension and upload the sitemaps to Web Scraper Cloud.

```

1 {
2   "_id": "c4l-journal",
3   "startUrl": [
4     "https://journal.code4lib.org/"
5   ],
6   "selectors": [
7     {
8       "id": "issue-title",
9       "multiple": false,
10      "parentSelectors": [
11        "_root"
12      ],
13      "regex": "",
14      "selector": "h1.pagetitle",
15      "type": "SelectorText"
16    },

```

Figure 3. Snippet of Web Scraper sitemap (see Appendix 1 for full example).

Web Scraper Cloud allows users to store sitemaps and create scrape job schedules unique to each sitemap. By default, the application supports downloading scraped data output as CSV, JSON, and XLSX. The scraped data can also be retrieved as JSON via the Web Scraper API, which was the preferred method to retrieve data for the recurring downloads pipeline. Web Scraper Cloud supports interfacing with other applications with a webhook that sends an HTTP POST request to a specified endpoint with a corresponding authentication token and a scrape job ID corresponding to the job that was kicked off.

```

1 // This is the data that your server receives
2 "scrapingjob_id": 1234
3 "status": "finished"
4 "sitemap_id": 12
5 "sitemap_name": "my-sitemap"
6 // Optional, custom_id will be passed to post data only if set

```

Figure 4. Web Scraper webhook data included in the HTTP POST request.

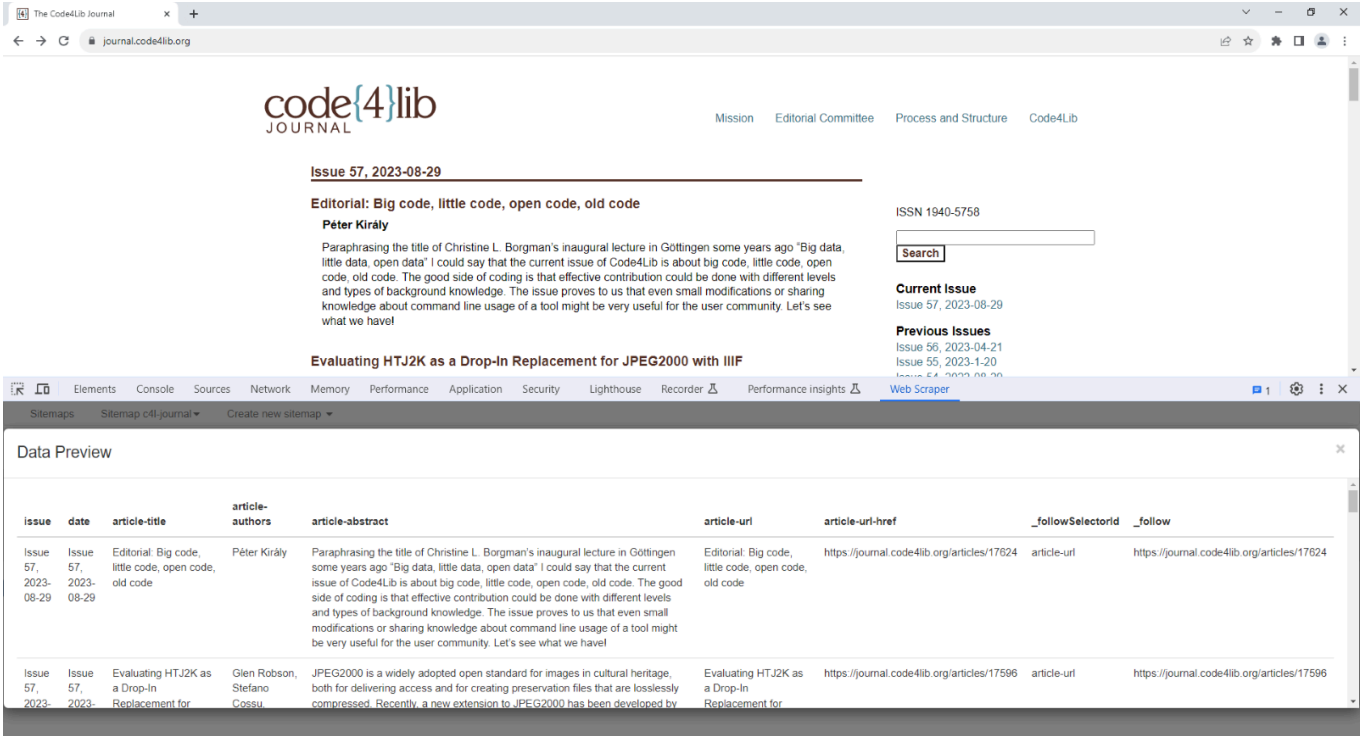


Figure 5. Screenshot of creating a sitemap to scrape articles from the current issue of Code4Lib Journal using the Web Scraper browser extension for Chrome.

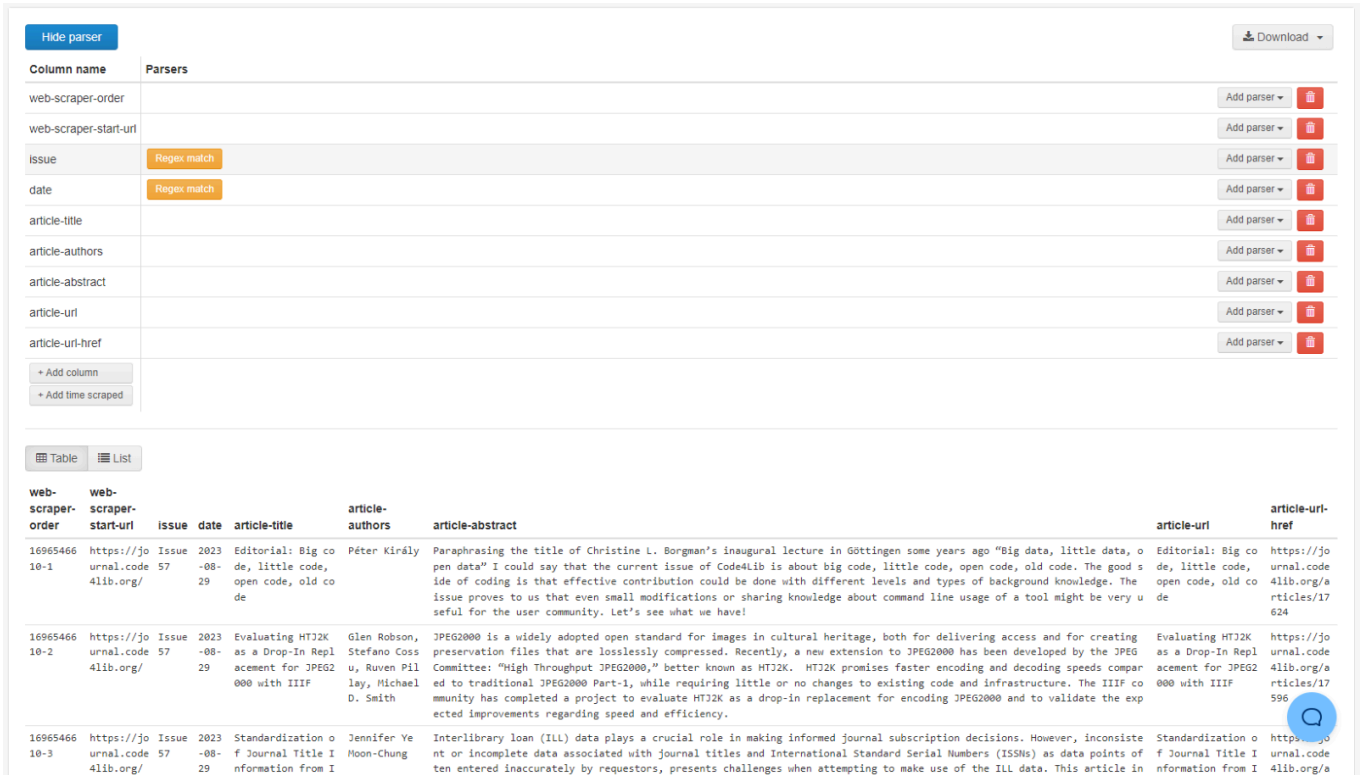


Figure 6. Screenshot of metadata preprocessing in Web Scraper Cloud.

Among the enhancements to the existing FRASER application, the development team created an HTTP endpoint to accept the POST data from Web Scraper and, if properly authenticated, invoke a serverless AWS Lambda instance with the scrape job ID as an input parameter. The Lambda then runs Python code that requests Web Scraper data corresponding to the scrape job ID, and performs metadata transformations and validation. The code queries AWS DynamoDB for a document that corresponds to the parent title record to which the scraped record would belong, and contains the specific transformations necessary to match the metadata payload that will eventually be inserted into the FRASER database, using pre-formatted variable inserts, such as date values and issue numbers. These injected variable keys are represented in the DynamoDB document, using a custom syntax developed for the project, with curly brackets used to represent variable names, such as {mm} for a zero-padded month number, and {month} for a month's proper name.

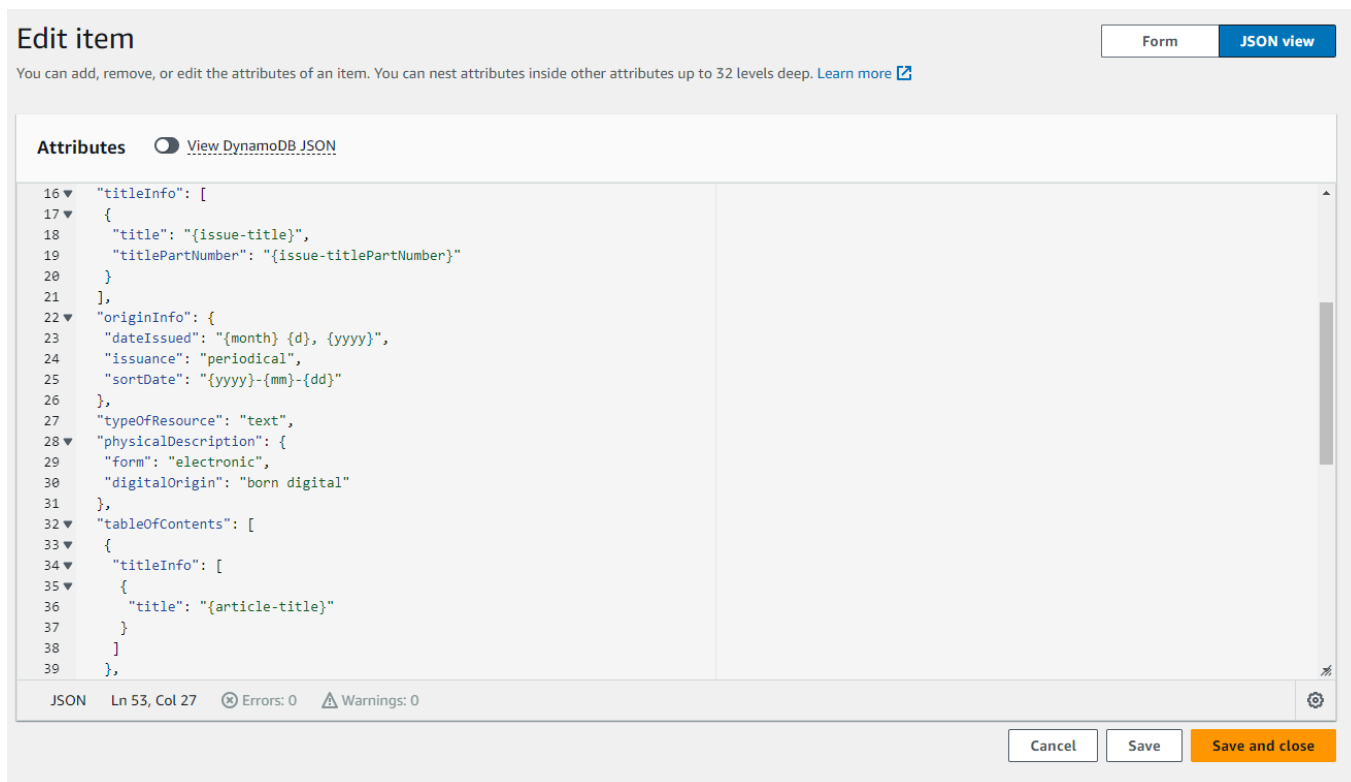


Figure 7. Screenshot of DynamoDB metadata template.

Among the metadata values generated by the Lambda is a filename that follows a standard naming convention, unique to each FRASER title, usually containing a timestamp value corresponding to the date the document was published. To determine whether the record already exists in the FRASER DAMS, the constructed filename is checked against the existing files in FRASER's file storage system in AWS Simple Storage Service (S3). If a file with the constructed name exists in S3, the file is considered not new, and the process terminates immediately. If a matching filename does not exist, the next step of the process kicks off. Checking only for new content in this way does not handle edge cases where a file has been revised with new or changed content but the filename remains the same. Several alternative options were considered but required exploration and experimentation that did not fit into the project timeline. Recommended future work would modify the process to check other unique criteria, such as a file checksum, in order to account for revised documents that would otherwise generate the same filename.

After file download and metadata transformation and validation completes, the Lambda uploads the document to FRASER file storage and inserts the metadata record into the FRASER relational database, where pending records are accessed through the FRASER DAMS. Once metadata and files reach the DAMS, digital library staff conduct a QA review of the document to ensure that the record contains correct metadata. In this manual step, staff are able to clean up records that may have missing or inaccurate metadata and, in cases where a document could not be retrieved through the automated process, manually capture the file.

Potential future enhancements for the process include advanced and filtered logging and tracking of the web scraping and ingest processes. Digital library staff have requested individual job logging in the DAMS dashboard to more easily track errors in the pipeline. Web Scraper records scrape job and webhook POST data, including scrape job information for records which, ultimately, were not ingested because the content already existed in the DAMS. The Web Scraper logs do not represent failures in the Lambda, where scraped content does not exist in FRASER but the process ceases for another reason. Notifications for and comprehensive logging of unsuccessful job runs would help staff quickly to address faulty runs in the future.

Challenges

From the outset of development to the final implementation, the development team encountered obstacles with automating the existing recurring downloads workflow. The manual process of archiving documents had been long fraught by issues with false-positive alerts from the website monitoring software and tedious, mundane daily tasks. The workflow, however, was clearly driven by institutional knowledge, not possible to replace fully with machine processes. Additionally, the development team met technical hurdles that slowed development and demanded revisiting and altering the planned architecture and workflow.

Several impediments that could have been anticipated and mitigated caused unanticipated disruptions.

Technology Stack

The developers supporting FRASER exclusively build and maintain PHP applications. The AWS Lambda service is limited in what languages are supported. The team quickly began work on the Lambda code using PHP and realized, after the bulk of the code had been completed, that Lambda would not support PHP in a straightforward way. Shifting from PHP to Python was a necessary transition and required members of the team to work in an unfamiliar language while development was already in progress. More thorough preliminary analysis could have caught this earlier and allowed time for staff training in the new language, or more thorough exploration of a solution that would have supported languages that the team was already familiar with.

While many of the born-digital documents are downloadable PDF files, some of the documents involved capturing semi-dynamic web pages as static PDF documents. To capture primarily textual content presented only as HTML, the manual archiving process depended on the print to PDF feature available in most modern web browsers. With several rounds of testing to automate this conversion process, Google Chrome's headless print option stood out as the best option for producing a PDF that held a look-and-feel most similar to the original web page. While similar functions exist in Python libraries, most still depend on a headless browser that emulates browser functionality without a graphical user interface (GUI). Python libraries that don't use a headless browser produce PDFs with varying levels of quality, and for the most part, lack the fidelity of the original web page present in browser-based methods. Using the headless browser necessitated including a Google Chrome binary with the deployed Lambda, which proved technically difficult using the infrastructure planned at the outset of the project. Because development was done locally on machines that had the Google Chrome binary and

due to the nature of user testing, this critical flaw in the pipeline was not realized until after the project was deployed to production, at which time rearchitecting the pipeline was no longer an option [4].

Institutional Limitations on Third-Party Software

Institutional requirements limited third-party software permissible for use. Web Scraper was a new software to the institution and was required to undergo thorough security audit and, as a cloud-based software, necessitated further evaluation and approval. The development team began implementation with the assumption Web Scraper would be approved for use. While the software ultimately was approved, plans to productionize the pipeline were dependent upon approval because a back-up solution that fully met the requirements set in the project planning had not been identified. Additionally, if OpenRefine had been ultimately pursued as a component of the pipeline architecture, it would have also been scrutinized by the internal review process.

Project Planning

Even with project planning, sufficient time was not allocated to planning and refining work and the iterative development and testing cycle needed for successfully working with the new toolkit. Areas of the workflow that required additional, more nuanced consideration were neglected. Many of the metadata transformation operations and templates were created ad-hoc, but, as a core function in the recurring downloads pipeline, this component of the workflow would have been greatly improved by standardizing a coherent schema, with corresponding documentation around the various metadata variables that would be scraped from the web sources. The lack of advance planning on metadata transformation led to confusion among the various stakeholders, including web developers and library staff, as to how the metadata transformation schema should be formatted, which led to on-the-fly decisions around basic requirements during development.

Pipeline Responsibilities and Sustainability

Early planning of the recurring downloads pipeline included considerations for library staff to have greater oversight of and responsibility for the pipeline. In the final product, the components that library staff maintained included the Web Scraper sitemaps and metadata templates stored in DynamoDB. While these components gave a majority of workflow ownership to library staff, the product was very advanced and technical. The pipeline would have benefited from an approach that leaned less on technical staff and more on open source tools and the knowledge of library staff. One such approach, that was abandoned early on in planning, was the use of OpenRefine.

The recurring downloads project was planned as a final business enhancement prior to a multi-year technical project that would leave applications in maintenance mode. Bugs and inadequate functionality that resulted from insufficient planning and user testing would not be in scope for development until after the technical project completion. Because the implementation of requirements were not fully designed as part of project planning, development would have required many iterations of testing and remediation.

The goal to automate collecting content from source websites, transforming and enhancing the metadata, and ingesting the metadata and files into the DAMS ultimately was a success. The FRASER team currently has a productionized workflow for gathering external metadata and documents and has ingested 500 new records through this process. False positive alerts, which previously took up a large portion of library staff time sifting through, have disappeared as a concern. In general, library staff time dedicated to the recurring download process has decreased. However, some of that time and effort has, in turn, shifted to technical staff, whose time has historically been more limited. Any institution looking to embark on an automation effort such as this would be advised to consider the job descriptions for staff that are responsible for supporting the process or function.

Conclusion

The FRASER library staff and web developers embarked on a formidable goal in the recurring downloads project. The ingenious plan would alleviate some of the workload and tedium involved in the daily manual process, while also optimizing the speed and efficiency with which new records could be added and increasing the scalability of collecting digital documents and publications. The project and resulting pipeline was successful in many ways: the amount of active time required to add new records was reduced – from getting an email notification with a URL, reviewing the site to ensure new content was present, adding files to the S3 file store and finally creating a metadata record, to simply clicking two buttons in a single location to QA and approve a record. The number of false positive alerts of new archivable documents was reduced to zero. Given constrained staff resources, the pipeline and simple dashboard made it possible to distribute the workload from a single employee to several staff without tedious training. However, the pipeline and its maintenance increased complexity for the recurring downloads workflow. Instead of digital library staff owning the archival process entirely, it became distributed between technical staff and library stakeholders. Much of the maintenance and additional work on the project had to be assumed by technical staff, whose capacity is consumed by other priorities outside of FRASER. While this project to automate and streamline contemporary digital publication collection was not without trials, it was a valuable learning experience for stakeholders involved, will be informative for future ventures to automate workflows and tasks, and ultimately created efficiencies for digital library staff.

Notes

[1] The first record created as part of what was later formalized as the recurring downloads workflow was the October 2006 issue of Economic Indicators <https://fraser.stlouisfed.org/title/1/item/134>.

[2] From 2020-2022 the sources in scope for collection and archiving increased from ~20 to ~50.

[3] FRASER includes [historical press releases from the U.S. Department of the Treasury](#) going back to 1916 and collected ongoing from the [U.S. Department of the Treasury Press Releases](#) website.

[4] A potential option would have been to use a container service such as Docker in order to provide greater flexibility in pulling in external dependencies for a codebase that is otherwise limited in Lambda.

About the Authors

Matthew Krc (he/him) is a Data Engineer and Product Owner of the FRED Data Desk team at the Federal Reserve Bank of St. Louis, where he contributes to and plans work on the data and metadata pipelines for the Federal Reserve Economic Data website. He has a professional background in library web application development and was previously part of the technical team that supports the FRASER application.

Anna Oates Schlaack (she/her) is a Product Owner in the Research Division at the Federal Reserve Bank of St. Louis, where she leads a web development team in modernizing legacy information discovery applications.

Appendices

Appendix 1. Example Web Scraper Sitemap

```

1  {
2    "_id": "c4l-journal",
3    "startUrl": [
4      "https://journal.code4lib.org/"
5    ],
6    "selectors": [
7      {
8        "id": "issue-title",
9        "multiple": false,
10       "parentSelectors": [
11         "_root"
12       ],
13       "regex": "",
14       "selector": "h1.pagetitle",
15       "type": "SelectorText"
16     },
17     {
18       "id": "issue-date",
19       "multiple": false,
20       "parentSelectors": [
21         "_root"
22       ],
23       "regex": "",
24       "selector": "h1.pagetitle",
25       "type": "SelectorText"
26     },
27     {
28       "id": "article-wrapper",
29       "multiple": true,
30       "parentSelectors": [
31         "_root"
32       ],
33       "selector": "div.article",
34       "type": "SelectorElement"
35     },
36     {
37       "id": "article-title",
38       "multiple": false,
39       "parentSelectors": [
40         "article-wrapper"
41       ],
42       "regex": "",
43       "selector": "a",
44       "type": "SelectorText"
45     },
46     {
47       "id": "article-authors",
48       "multiple": false,
49       "parentSelectors": [
50         "article-wrapper"
51       ],
52       "regex": "",
53       "selector": "p.author",
54       "type": "SelectorText"
55     },
56     {
57       "id": "article-abstract",
58       "multiple": false,
59       "parentSelectors": [
60         "article-wrapper"
61       ],
62       "regex": "",
63       "selector": ".abstract p",
64       "type": "SelectorText"
65     },
66     {
67       "id": "article-url",
68       "linkType": "linkFromHref",
69       "multiple": false,
70       "parentSelectors": [
71         "article-wrapper"
72       ],
73       "selector": "a",
74       "type": "SelectorLink"
75     }
76   ]
77 }

```

Appendix 2. Example AWS DynamoDB Document

```

1  {
2    "id": "c4l-journal",
3    "sortkey": "c4l-journal",
4    "cron": "0 22 * 2,5,8,11 5",
5    "filenames": {
6      "pdf": "c4l_{yyyy}_{nn}.pdf"
7    },
8    "frequency": "q",
9    "metadata": {
10     "genre": [
11       "periodical"

```

```

12     ],
13     "language": [
14         "eng"
15     ],
16     "titleInfo": [
17         {
18             "title": "{issue-title}",
19             "titlePartNumber": "{issue-titlePartNumber}"
20         }
21     ],
22     "originInfo": {
23         "dateIssued": "{month} {d}, {yyyy}",
24         "issuance": "periodical",
25         "sortDate": "{yyyy}-{mm}-{dd}"
26     },
27     "typeOfResource": "text",
28     "physicalDescription": {
29         "form": "electronic",
30         "digitalOrigin": "born digital"
31     },
32     "tableOfContents": [
33         {
34             "titleInfo": [
35                 {
36                     "title": "{article-title}"
37                 }
38             ]
39         },
40         {
41             "name": [
42                 {
43                     "role": "creator",
44                     "namePart": [
45                         "{article-author}"
46                     ]
47                 }
48             ]
49         }
50     ],
51     "relatedItem": {
52         "1": {
53             "@type": "itemParent",
54             "recordInfo": {
55                 "recordIdentifier": [
56                     "123"
57                 ]
58             },
59             "titleInfo": [
60                 {
61                     "title": "Code4Lib Journal"
62                 }
63             ]
64         }
65     },
66 },
67 "s3_path": "/docs/publications/c4l/"
68 }

```

Subscribe to comments: [For this article](#) | [For all articles](#)

This work is licensed under a Creative Commons Attribution 3.0 United States License.

